Workshop No. 3: Robust System Design and Project Management

Equipment and Connectivity Management (ECM) Platform

Daniel Martínez Gil, Dylan Silva Orrego, Juan Ramírez Hérnandez, Juan Triana Paipa
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
dammartinezg@udistrital.edu.co, jgramirezh@udistrital.edu.co, ddsilvao@udistrital.edu.co, jctrianap@udistrital.edu.co

***Abstract*—The transition from conceptual modeling to a robust software engineering framework requires a comprehensive approach to system architecture and project execution. This document details the architectural evolution of the Equipment and Connectivity Management (ECM) platform, integrating rigorous quality standards, proactive risk mitigation, and structured management controls to support an academic community of 7,000 potential users. The outcomes establish a production-ready blueprint that guarantees system scalability, regulatory compliance, and operational sustainability before full-scale implementation.**

***Index Terms*—System Design Refinement, Clean Architecture, Risk Management, Project Governance, Robust Engineering**

I. Enhanced System Design Document

This section presents the comprehensive system design, transitioning the ECM platform from conceptual models to a production-ready architectural blueprint.

A. Executive Summary

The ECM platform development has evolved significantly from its initial requirement gathering phases. This summary provides a comprehensive overview of the design evolution, highlighting the integration of quality enhancements and a structured management approach to support an academic community of 7,000 users.

B. Robust Architecture Design

Building upon the Clean Architecture paradigm, the updated architectural documentation incorporates quality standards (ISO/IEC 25010) and engineering best practices. The core domain logic, implemented in Java, is decoupled from the persistence and presentation layers to ensure long-term maintainability and fault tolerance under high concurrent loads.

C. Risk Management Plan

Following the ISO 31000 framework, this detailed risk analysis identifies vulnerabilities and proposes monitoring approaches and mitigation strategies to protect institutional operational continuity.

TABLE I: Risk Management Matrix for the ECM Platform.

ID	Risk Description	Prob.	Imp.
R01	Server downtime during peak academic concurrent user access.	Med	High
R02	Data inconsistency during simultaneous equipment lending requests.	Low	High

D. Quality Assurance Framework

To guarantee system reliability, this framework defines quality metrics, validation methods, and acceptance criteria. The backend Java ecosystem will enforce a minimum of 80% unit test coverage for the lending domain logic.

E. Implementation Strategy

The deployment of the ECM platform will follow a phased approach, detailing the timeline and resource requirements. Initial rollout will consist of a controlled pilot with institutional technical staff before a university-wide release.

F. Evolution Summary

This subsection documents the design improvements and lessons learned from the previous workshops. The shift towards a dynamic Observer pattern for notifications was a direct result of user feedback regarding inventory visibility.

II. Project Management Documentation

This section establishes the governance, structural composition, and operational controls required to manage the transition of the ECM platform into a functional, production-ready system.

A. Project Charter

The Project Charter formally authorizes the ECM platform implementation to replace legacy manual lending for 7,000 potential users. The project scope encompasses a decoupled multi-tier architecture, strict institutional lending policy enforcement, and secure user data management. Success is defined by the complete elimination of concurrency conflicts and a system response time under two seconds.

B. Team Structure and Roles

The engineering team is organized under a strict accountability framework divided into four specialized domains. Project Management oversees stakeholder alignment and regulatory compliance; Backend Engineering implements the core Java lending logic and transaction locks; Frontend Engineering develops the responsive React user interface; and Quality Assurance designs automated testing pipelines to enforce an 80% unit test coverage.

C. Project Timeline

The implementation lifecycle follows a phased progression designed to minimize technical debt and ensure thorough validation. Execution begins with environment and repository configuration, moves to core Java domain codification, and proceeds to frontend-backend API integration. The schedule concludes with a controlled pilot deployment among technical operators before the final university-wide rollout.

D. Resource Management Plan

Resource allocation is managed dynamically to maximize development capacity and infrastructural efficiency. Human resources are balanced across development layers to prevent integration bottlenecks, while technical planning provisions cloud hosting and continuous integration pipelines. Capacity forecasting guarantees that database and server infrastructure can handle peak academic concurrent traffic without performance degradation.

E. Communication and Control Plan

A rigorous communication framework ensures transparency and alignment among all institutional stakeholders. Monitoring relies on weekly technical reviews to assess milestone completion and address architectural friction early. Project control mechanisms utilize automated tracking tools against the planned baseline, generating periodic progress and quality reports for university administrative approval.

III. Visual Representations and Tools

This section presents the complete set of visual artifacts that support the ECM platform design, risk management, project planning, and quality assurance processes. All diagrams were produced with TikZ/PGF inside LaTeX to guarantee reproducibility and version-control compatibility within the project repository.

A. Enhanced System Architecture Diagram

The diagram below reflects the robust, layered architecture of the ECM platform built upon Clean Architecture principles (ISO/IEC 25010). Three external actor groups interact with three internal tiers — Presentation, Application Logic, and Data/Integration — through clearly defined interface boundaries.

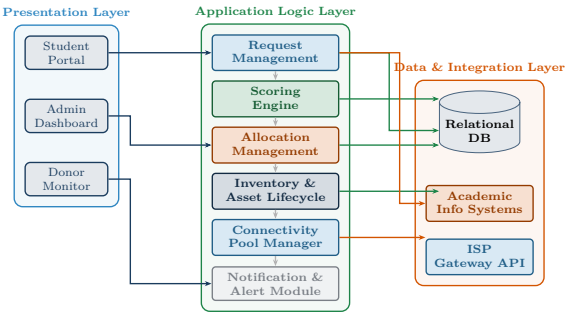


Fig. 1: Enhanced ECM System Architecture Diagram. Dashed arrows denote internal module data flow; solid arrows denote actor-to-layer and layer-to-layer communication.

B. Risk Management Matrix Visualization

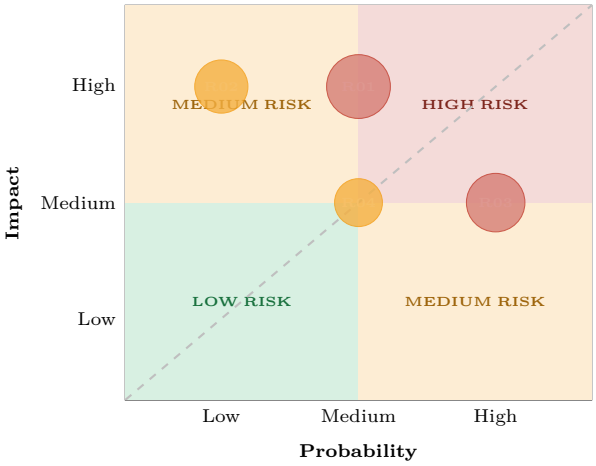
The bubble chart below maps the four identified risks on a Probability × Impact grid. Bubble area encodes overall risk severity (Probability × Impact on a 1–3 ordinal scale). Risks that fall in the upper-right quadrant require immediate mitigation.

C. Quality Assurance Process Flow

The flowchart below depicts the three-tier QA validation framework applied to every software increment of the ECM platform, from unit testing through production monitoring. Rework loops ensure that no increment advances until its tier acceptance criteria are met.

IV. Results & Discussion

The refinement of the architecture demonstrates a structural improvement over the baseline models. Decoupling the domain layer ensures that future updates to database vendor engines or frontend components will not introduce regressions into the validated business logic. The inclusion of predictive capacity tracking further guarantees that resource constraints are identified before impacting service availability.



- **R01** — Server downtime during peak academic hours.
- **R02** — Data inconsistency in simultaneous lending.
- **R03** — Frontend-backend integration delays.
- **R04** — Low user adoption of the platform.

Fig. 2: Risk Management Bubble Chart — Probability vs. Impact.

V. Conclusions

The conceptual and management frameworks developed across these initial phases provide a scalable, sustainable path toward automation. By anchoring design decisions in empirical user data and standardizing project controls, the proposed blueprint bridges the gap between academic systems design and reliable industrial implementation.

Acknowledgments

The author thanks the Computer Engineering Program faculty at Universidad Distrital Francisco José de Caldas for the methodological guidelines provided.

References

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA: Addison-Wesley, 2021.

[2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, 1st ed. Boston, MA: Prentice Hall, 2017.

[3] International Organization for Standardization, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (Square) — System and software quality models*, ISO/IEC Standard 25010:2011, 2011.

[4] International Organization for Standardization, *Risk management — Guidelines*, ISO Standard 31000:2018, 2018.

[5] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA: Project Management Institute, 2021.

[6] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, 1st ed. Boston, MA: Addison-Wesley, 2003.

[7] C. Walls, *Spring in Action*, 6th ed. Shelter Island, NY: Manning Publications, 2022.

[8] A. Banks and Eve Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2023.

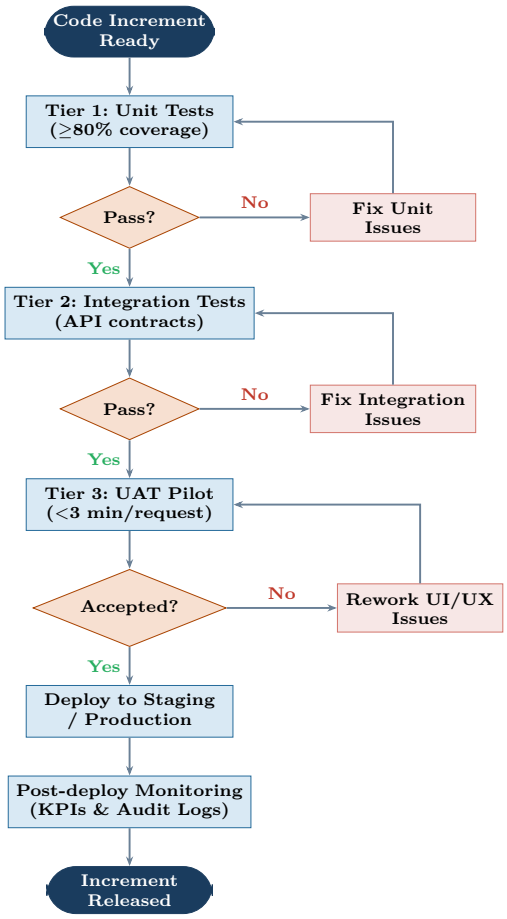


Fig. 3: ECM Quality Assurance Process Flow — Three-tier validation strategy with rework loops.